

Java Logic-Building Prompt Pack

9 master prompts x 100 questions = 900 practice questions, built to turn Java fundamentals into problem-solving ability.

Java	Logic Building	DSA	Core CS	Projects	Placements
------	----------------	-----	---------	----------	------------

You are here: Java fundamentals -> Logic Building. Advanced DSA comes later, so these prompts deliberately avoid it.

What is inside

#	Master topic	Question mix	What it builds
1	Java Basics	Conceptual + MCQ + predict output + find the error	Builds the mental model of what actually happens between writing code and seeing output.
2	Variables, Data Types, Keywords and Identifiers	Predict output + short programs + compile errors + traps	Builds precision: knowing exactly what Java stores, converts and rejects.
3	Input and Output in Java	Input-process-output programs + printf formatting + mini tasks	Builds the input -> process -> output habit that every problem starts with.
4	Operators	Predict output + expressions + bitwise + ternary + bug hunts	Builds expression-level thinking: exactly how Java evaluates one line.
5	Flow Control	Decision programs + switch + real-world rules + boundary testing	Builds decision-making logic: turning rules and boundaries into conditions.
6	Loops	Dry-run tracing + number logic + patterns + do-while	Builds repetition logic: the engine behind most problem solving.
7	Methods	Writing methods + overloading + recursion tracing + decomposition	Builds decomposition: splitting a problem into small, reusable, testable pieces.
8	Array	Trace + 1D problems + searching/sorting basics + 2D + bug hunts	Builds data-handling logic: index thinking, traversal and in-place updates.
9	String	Trace + String API + manipulation + StringBuilder + logic problems	Builds text-processing logic and character-level thinking.

How to use this pack

1. Open a fresh chat with a strong AI model for each topic. Copy the entire prompt from the teal-topped box.
2. The AI sends 25 questions at a time. Type **next** to get the following batch. Asking for 25 at a time keeps quality high; a single 100-question reply degrades.
3. Solve in order. Do not skip Level 1, even if it feels easy. Weak logic usually comes from skipped fundamentals.
4. Use this loop for every question: **read, rephrase** in your own words, **work 2 samples by hand, write the steps in plain English, code, test edge cases**. Only then check an answer.
5. Struggle for 20 to 30 minutes before asking for help, and use the hint ladder in the appendix instead of asking for the full solution.
6. Suggested pace: 10 questions a day. Topics 1 to 3 are lighter and can go faster; Topics 5, 6, 7 and 8 deserve the most time.
7. AI-written sample outputs can occasionally be wrong. If a sample output looks off, run the code yourself in Java. Treat it as extra practice in verifying.

Java Basics

Builds the mental model of what actually happens between writing code and seeing output.

PROMPT: COPY EVERYTHING IN THE BOX BELOW INTO A NEW CHAT

ROLE

You are a senior Java instructor and campus-placement coach. You specialise in helping students who already know Java syntax but cannot turn a problem statement into working logic.

LEARNER

A 3rd-year engineering student whose logic building is weak. Roadmap: Java -> Logic Building -> DSA -> Core CS -> Projects -> Placements. We are at the Java fundamentals stage and are NOT doing advanced DSA yet. The questions must convert Java fundamentals into problem-solving ability, not test memorisation of syntax.

TASK

Generate exactly 100 high-quality practice questions on the master topic "Java Basics".

SUBTOPICS TO COVER (each in at least 3 questions)

- What Java is and why it is called platform independent
- JDK vs JRE vs JVM (who does what)
- Bytecode and the compile-and-run cycle (javac, java)
- Program structure: class, main method, every keyword in public static void main(String[] args)
- File name vs public class name rule
- Comments (single-line, multi-line, documentation)
- Naming conventions for classes, methods, variables, constants
- print vs println, escape sequences (\n, \t, \"), compile-time vs run-time errors

ALLOWED CONCEPTS

Only this topic.

NOT ALLOWED

Scanner, variables, conditions, loops, arrays, user-defined methods, classes other than the main class, collections. Programs may only print literal text.

QUESTION MIX (total must be exactly 100)

- 35 questions: conceptual short-answer (why / what / difference), answerable in 2 to 4 lines
- 15 questions: MCQ with 4 options, where the wrong options are common student misconceptions
- 20 questions: predict the output, or 'does this compile? why?' (print vs println, escape sequences, comments hiding code)
- 15 questions: find and fix the error (missing semicolon, wrong main signature, class/file mismatch, case sensitivity)
- 10 questions: write a small program (name card, text box, multi-line message using \n and \t)
- 5 questions: 'explain it to a friend' interview-style (why is main static? why is Java platform independent?)

DIFFICULTY RAMP (mix the types across the levels)

- Level 1 Warm-up (Q1-Q30): one concept, direct application.
- Level 2 Core (Q31-Q70): two ideas combined; needs a small plan on paper.
- Level 3 Stretch (Q71-Q90): needs a plan plus edge-case thinking.
- Level 4 Interview flavour (Q91-Q100): classic questions from campus tests; brute force is still fine.

RULES FOR EVERY QUESTION

1. Each question must be fully self-contained: a complete problem statement, input format, output format, constraints and at least one sample input/output (for coding questions). For predict-the-output, MCQ and conceptual questions show the code or scenario instead.
2. One core idea per question. Order the whole set from easy to hard.
3. No near-duplicates. Do not repeat the same idea with only different numbers; vary the real-world context (shop, marks, bank, game, time, temperature, travel).
4. Constraints must be small so that the simplest brute-force solution works. Never demand optimisation.
5. Before including a question, silently trace every sample output and every predict-the-output answer step by step. If you are not fully sure about Java's behaviour, drop the question and write another. Do not print the trace.
6. Cover edge cases on purpose: zero, negative values, boundary values, maximum values, empty or single-element input.

7. Use plain English and Java 17 with standard classes only. The difficulty must come from the logic, never from confusing wording.
8. Do NOT give solutions, code answers or output answers. For Level 2, 3 and 4 questions add a one-line 'Thinking hint' that nudges the approach without revealing the algorithm.
9. Tag every question with its number, type, level and the concept it tests.
10. After Q100, print a coverage checklist: each subtopic above with the question numbers that cover it (at least 3 each).
11. Conceptual questions must have one clearly correct short answer. Do not ask for opinions.

OUTPUT FORMAT (use this layout for every question)

Q12 | Type: Program | Level 2 (Core) | Concept: digit extraction

Problem: (full statement)

Input: (format)

Output: (format)

Sample Input: ...

Sample Output: ...

Thinking hint: (Level 2 and above only)

DELIVERY

Do not write all 100 at once. Send four batches: Q1-25, Q26-50, Q51-75, Q76-100. After each batch stop and wait until I type 'next'. After batch 4, print the coverage checklist. Give answer keys only when I ask for them.

Variables, Data Types, Keywords and Identifiers

Builds precision: knowing exactly what Java stores, converts and rejects.

PROMPT: COPY EVERYTHING IN THE BOX BELOW INTO A NEW CHAT

ROLE

You are a senior Java instructor and campus-placement coach. You specialise in helping students who already know Java syntax but cannot turn a problem statement into working logic.

LEARNER

A 3rd-year engineering student whose logic building is weak. Roadmap: Java -> Logic Building -> DSA -> Core CS -> Projects -> Placements. We are at the Java fundamentals stage and are NOT doing advanced DSA yet. The questions must convert Java fundamentals into problem-solving ability, not test memorisation of syntax.

TASK

Generate exactly 100 high-quality practice questions on the master topic "Variables, Data Types, Keywords and Identifiers".

SUBTOPICS TO COVER (each in at least 3 questions)

- Declaring, initialising and assigning variables
- The 8 primitive types: size, range, default values (byte, short, int, long, float, double, char, boolean)
- Primitive vs reference types (String as a basic reference type)
- Literals: long L suffix, float f suffix, char, hex/binary/octal, underscores
- Implicit widening and explicit narrowing (type casting), overflow, precision loss
- char and ASCII arithmetic
- final constants
- Keywords vs identifiers, valid and invalid identifiers
- Variable scope (local, block) and redeclaration errors

ALLOWED CONCEPTS

This topic plus everything from the earlier topics: Java Basics.

NOT ALLOWED

Scanner input (use hard-coded values), conditions, loops, arrays, user-defined methods, OOP, collections. Only the arithmetic operators + - * / % may be used; String only as text and concatenation.

QUESTION MIX (total must be exactly 100)

- 25 questions: predict the output (casting, overflow, char + int, int / int, 0.1 + 0.2, long literals, byte + byte promotion)
- 15 questions: valid or invalid? identifiers, literals and declarations, with the reason
- 25 questions: short programs with hard-coded values (swap two variables, paise to rupees, Celsius to Fahrenheit, simple interest, average of 3, char to ASCII, seconds to h:m:s, extract digits of a fixed 3-digit number using / and %)
- 15 questions: find and fix compile errors (possible lossy conversion, uninitialised variable, out of scope, redeclaration, reassigning a final)
- 10 questions: conceptual (primitive vs reference, default values, why float needs f, why long needs L)
- 10 questions: trap questions (byte b = 10; b = b + 1; / Integer.MAX_VALUE + 1 / 'a' + 'b' / 5 / 2 * 2.0)

DIFFICULTY RAMP (mix the types across the levels)

- Level 1 Warm-up (Q1-Q30): one concept, direct application.
- Level 2 Core (Q31-Q70): two ideas combined; needs a small plan on paper.
- Level 3 Stretch (Q71-Q90): needs a plan plus edge-case thinking.
- Level 4 Interview flavour (Q91-Q100): classic questions from campus tests; brute force is still fine.

RULES FOR EVERY QUESTION

1. Each question must be fully self-contained: a complete problem statement, input format, output format, constraints and at least one sample input/output (for coding questions). For predict-the-output, MCQ and conceptual questions show the code or scenario instead.
2. One core idea per question. Order the whole set from easy to hard.
3. No near-duplicates. Do not repeat the same idea with only different numbers; vary the real-world context (shop, marks, bank, game, time, temperature, travel).
4. Constraints must be small so that the simplest brute-force solution works. Never demand optimisation.
5. Before including a question, silently trace every sample output and every predict-the-output answer step by step. If you are not fully sure about Java's behaviour, drop the question and write another. Do not print the trace.

6. Cover edge cases on purpose: zero, negative values, boundary values, maximum values, empty or single-element input.
7. Use plain English and Java 17 with standard classes only. The difficulty must come from the logic, never from confusing wording.
8. Do NOT give solutions, code answers or output answers. For Level 2, 3 and 4 questions add a one-line 'Thinking hint' that nudges the approach without revealing the algorithm.
9. Tag every question with its number, type, level and the concept it tests.
10. After Q100, print a coverage checklist: each subtopic above with the question numbers that cover it (at least 3 each).
11. Predict-the-output snippets must be 3 to 8 lines and show every declaration.
12. For at least 10 questions add a follow-up: 'Explain why the result is not what a beginner expects.'

OUTPUT FORMAT (use this layout for every question)

Q12 | Type: Program | Level 2 (Core) | Concept: digit extraction

Problem: (full statement)

Input: (format)

Output: (format)

Sample Input: ...

Sample Output: ...

Thinking hint: (Level 2 and above only)

DELIVERY

Do not write all 100 at once. Send four batches: Q1-25, Q26-50, Q51-75, Q76-100. After each batch stop and wait until I type 'next'. After batch 4, print the coverage checklist. Give answer keys only when I ask for them.

Input and Output in Java

Builds the input -> process -> output habit that every problem starts with.

PROMPT: COPY EVERYTHING IN THE BOX BELOW INTO A NEW CHAT

ROLE

You are a senior Java instructor and campus-placement coach. You specialise in helping students who already know Java syntax but cannot turn a problem statement into working logic.

LEARNER

A 3rd-year engineering student whose logic building is weak. Roadmap: Java -> Logic Building -> DSA -> Core CS -> Projects -> Placements. We are at the Java fundamentals stage and are NOT doing advanced DSA yet. The questions must convert Java fundamentals into problem-solving ability, not test memorisation of syntax.

TASK

Generate exactly 100 high-quality practice questions on the master topic "Input and Output in Java".

SUBTOPICS TO COVER (each in at least 3 questions)

- System.out.print, println, printf, String.format
- Format specifiers: %d %f %.2f %s %c %b %n, width, left-align, zero padding
- Escape sequences
- Scanner setup: import, new Scanner(System.in), closing
- nextInt, nextDouble, next, nextLine, nextBoolean, reading a char with next().charAt(0)
- The nextInt() followed by nextLine() newline trap
- Reading several values on one line, mixed-type input
- Simple input-based calculations

ALLOWED CONCEPTS

This topic plus everything from the earlier topics: Java Basics; Variables, Data Types, Keywords and Identifiers.

NOT ALLOWED

Conditions, loops, arrays, user-defined methods, OOP, collections. Because loops are not covered yet, every program reads a FIXED number of inputs.

QUESTION MIX (total must be exactly 100)

- 45 questions: input -> compute -> output programs (bill split, simple interest, BMI, area of shapes, discount price, time conversion, average, fuel cost)
- 15 questions: formatted-output tasks (receipts, tables, alignment, exact decimals) with the exact expected output shown
- 10 questions: mixed-type input (int + String + double + char), including the nextLine pitfall
- 10 questions: predict the output for printf / println with specifiers and escape sequences
- 10 questions: find and fix (InputMismatchException cause, wrong specifier, skipped nextLine, missing import)
- 10 questions: sequential mini-projects (report card, ATM slip, electricity bill slip, ticket printer) with no decisions

DIFFICULTY RAMP (mix the types across the levels)

- Level 1 Warm-up (Q1-Q30): one concept, direct application.
- Level 2 Core (Q31-Q70): two ideas combined; needs a small plan on paper.
- Level 3 Stretch (Q71-Q90): needs a plan plus edge-case thinking.
- Level 4 Interview flavour (Q91-Q100): classic questions from campus tests; brute force is still fine.

RULES FOR EVERY QUESTION

1. Each question must be fully self-contained: a complete problem statement, input format, output format, constraints and at least one sample input/output (for coding questions). For predict-the-output, MCQ and conceptual questions show the code or scenario instead.
2. One core idea per question. Order the whole set from easy to hard.
3. No near-duplicates. Do not repeat the same idea with only different numbers; vary the real-world context (shop, marks, bank, game, time, temperature, travel).
4. Constraints must be small so that the simplest brute-force solution works. Never demand optimisation.
5. Before including a question, silently trace every sample output and every predict-the-output answer step by step. If you are not fully sure about Java's behaviour, drop the question and write another. Do not print the trace.
6. Cover edge cases on purpose: zero, negative values, boundary values, maximum values, empty or single-element input.

7. Use plain English and Java 17 with standard classes only. The difficulty must come from the logic, never from confusing wording.
8. Do NOT give solutions, code answers or output answers. For Level 2, 3 and 4 questions add a one-line 'Thinking hint' that nudges the approach without revealing the algorithm.
9. Tag every question with its number, type, level and the concept it tests.
10. After Q100, print a coverage checklist: each subtopic above with the question numbers that cover it (at least 3 each).
11. Show Sample Input and Sample Output as separate labelled blocks. For printf questions show the exact expected output including spaces.

OUTPUT FORMAT (use this layout for every question)

Q12 | Type: Program | Level 2 (Core) | Concept: digit extraction

Problem: (full statement)

Input: (format)

Output: (format)

Sample Input: ...

Sample Output: ...

Thinking hint: (Level 2 and above only)

DELIVERY

Do not write all 100 at once. Send four batches: Q1-25, Q26-50, Q51-75, Q76-100. After each batch stop and wait until I type 'next'. After batch 4, print the coverage checklist. Give answer keys only when I ask for them.

Operators

Builds expression-level thinking: exactly how Java evaluates one line.

PROMPT: COPY EVERYTHING IN THE BOX BELOW INTO A NEW CHAT

ROLE

You are a senior Java instructor and campus-placement coach. You specialise in helping students who already know Java syntax but cannot turn a problem statement into working logic.

LEARNER

A 3rd-year engineering student whose logic building is weak. Roadmap: Java -> Logic Building -> DSA -> Core CS -> Projects -> Placements. We are at the Java fundamentals stage and are NOT doing advanced DSA yet. The questions must convert Java fundamentals into problem-solving ability, not test memorisation of syntax.

TASK

Generate exactly 100 high-quality practice questions on the master topic "Operators".

SUBTOPICS TO COVER (each in at least 3 questions)

- Arithmetic operators, integer division, % with negative numbers
- Relational and logical operators, short-circuit && and ||
- Assignment and compound assignment (+, -=, *=, /=, %=), implicit cast in compound assignment
- Unary, increment and decrement (pre vs post)
- Ternary operator, nested ternary
- Bitwise operators (& | ^ ~) and shift operators (<< >> >>>)
- Operator precedence and associativity, use of parentheses
- String concatenation with +

ALLOWED CONCEPTS

This topic plus everything from the earlier topics: Java Basics; Variables, Data Types, Keywords and Identifiers; Input and Output in Java.

NOT ALLOWED

if/switch, loops, arrays, user-defined methods, OOP, collections (the ternary operator IS allowed because it is this topic). Programs read a fixed number of inputs or use hard-coded values.

QUESTION MIX (total must be exactly 100)

- 30 questions: predict the output (precedence, associativity, i++ vs ++i inside expressions, compound assignment, integer division, short-circuit, = vs ==, string + number)
- 25 questions: write expressions or short programs (even/odd, divisibility, fixed-digit extraction with / and %, swap without a temp variable, leap year as ONE boolean expression, largest of 3 using ternary)
- 20 questions: bitwise and shift (check even/odd with &, multiply/divide by powers of 2, k-th bit check, set/clear/toggle a bit, power-of-two check, XOR swap)
- 10 questions: ternary (nested ternary, rewrite an if-else as ternary and back)
- 10 questions: find and fix logic bugs (= vs ==, missing parentheses, integer division bug)
- 5 questions: express a real-world rule as one boolean expression (eligibility, discount, access rule)

DIFFICULTY RAMP (mix the types across the levels)

- Level 1 Warm-up (Q1-Q30): one concept, direct application.
- Level 2 Core (Q31-Q70): two ideas combined; needs a small plan on paper.
- Level 3 Stretch (Q71-Q90): needs a plan plus edge-case thinking.
- Level 4 Interview flavour (Q91-Q100): classic questions from campus tests; brute force is still fine.

RULES FOR EVERY QUESTION

1. Each question must be fully self-contained: a complete problem statement, input format, output format, constraints and at least one sample input/output (for coding questions). For predict-the-output, MCQ and conceptual questions show the code or scenario instead.
2. One core idea per question. Order the whole set from easy to hard.
3. No near-duplicates. Do not repeat the same idea with only different numbers; vary the real-world context (shop, marks, bank, game, time, temperature, travel).
4. Constraints must be small so that the simplest brute-force solution works. Never demand optimisation.
5. Before including a question, silently trace every sample output and every predict-the-output answer step by step. If you are not fully sure about Java's behaviour, drop the question and write another. Do not print the trace.

6. Cover edge cases on purpose: zero, negative values, boundary values, maximum values, empty or single-element input.
7. Use plain English and Java 17 with standard classes only. The difficulty must come from the logic, never from confusing wording.
8. Do NOT give solutions, code answers or output answers. For Level 2, 3 and 4 questions add a one-line 'Thinking hint' that nudges the approach without revealing the algorithm.
9. Tag every question with its number, type, level and the concept it tests.
10. After Q100, print a coverage checklist: each subtopic above with the question numbers that cover it (at least 3 each).
11. For bitwise questions show operands in 8-bit binary in the sample so the learner can trace bit by bit. Keep all values small.

OUTPUT FORMAT (use this layout for every question)

Q12 | Type: Program | Level 2 (Core) | Concept: digit extraction

Problem: (full statement)

Input: (format)

Output: (format)

Sample Input: ...

Sample Output: ...

Thinking hint: (Level 2 and above only)

DELIVERY

Do not write all 100 at once. Send four batches: Q1-25, Q26-50, Q51-75, Q76-100. After each batch stop and wait until I type 'next'. After batch 4, print the coverage checklist. Give answer keys only when I ask for them.

Flow Control

Builds decision-making logic: turning rules and boundaries into conditions.

PROMPT: COPY EVERYTHING IN THE BOX BELOW INTO A NEW CHAT

ROLE

You are a senior Java instructor and campus-placement coach. You specialise in helping students who already know Java syntax but cannot turn a problem statement into working logic.

LEARNER

A 3rd-year engineering student whose logic building is weak. Roadmap: Java -> Logic Building -> DSA -> Core CS -> Projects -> Placements. We are at the Java fundamentals stage and are NOT doing advanced DSA yet. The questions must convert Java fundamentals into problem-solving ability, not test memorisation of syntax.

TASK

Generate exactly 100 high-quality practice questions on the master topic "Flow Control".

SUBTOPICS TO COVER (each in at least 3 questions)

- if, if-else, else-if ladder, nested if
- Combining conditions with && || !
- switch on int, char and String; break, default, fall-through
- Ternary as a decision tool
- Order of conditions and overlapping ranges
- Dangling else
- Boundary values and edge cases
- Choosing between if-else and switch

ALLOWED CONCEPTS

This topic plus everything from the earlier topics: Java Basics; Variables, Data Types, Keywords and Identifiers; Input and Output in Java; Operators.

NOT ALLOWED

Loops, arrays, user-defined methods, OOP, collections. Programs read a fixed number of inputs.

QUESTION MIX (total must be exactly 100)

- 15 questions: predict the output (else-if order, switch fall-through, missing break, dangling else, nested if)
- 30 questions: basic decision programs (sign check, largest of 3, leap year, vowel or consonant, grade calculator, triangle validity and type, day name, character category, nature of quadratic roots)
- 20 questions: switch-based programs (calculator, days in a month, menu with a single choice, vowel check via fall-through)
- 20 questions: real-world rule programs (income-tax slabs, electricity-bill slabs, shipping charges, movie-ticket pricing, admission eligibility, ATM withdrawal validity)
- 10 questions: find and fix logic bugs (wrong condition order, overlapping ranges, = vs ==, missing braces)
- 5 questions: boundary-value questions: give a program and ask for a set of test inputs that covers every branch

DIFFICULTY RAMP (mix the types across the levels)

- Level 1 Warm-up (Q1-Q30): one concept, direct application.
- Level 2 Core (Q31-Q70): two ideas combined; needs a small plan on paper.
- Level 3 Stretch (Q71-Q90): needs a plan plus edge-case thinking.
- Level 4 Interview flavour (Q91-Q100): classic questions from campus tests; brute force is still fine.

RULES FOR EVERY QUESTION

1. Each question must be fully self-contained: a complete problem statement, input format, output format, constraints and at least one sample input/output (for coding questions). For predict-the-output, MCQ and conceptual questions show the code or scenario instead.
2. One core idea per question. Order the whole set from easy to hard.
3. No near-duplicates. Do not repeat the same idea with only different numbers; vary the real-world context (shop, marks, bank, game, time, temperature, travel).
4. Constraints must be small so that the simplest brute-force solution works. Never demand optimisation.
5. Before including a question, silently trace every sample output and every predict-the-output answer step by step. If you are not fully sure about Java's behaviour, drop the question and write another. Do not print the trace.
6. Cover edge cases on purpose: zero, negative values, boundary values, maximum values, empty or single-element input.

7. Use plain English and Java 17 with standard classes only. The difficulty must come from the logic, never from confusing wording.
8. Do NOT give solutions, code answers or output answers. For Level 2, 3 and 4 questions add a one-line 'Thinking hint' that nudges the approach without revealing the algorithm.
9. Tag every question with its number, type, level and the concept it tests.
10. After Q100, print a coverage checklist: each subtopic above with the question numbers that cover it (at least 3 each).
11. For rule-based questions state every slab and boundary exactly (for example 'up to and including 5000') so there is only one correct answer.
12. At least 10 questions must be decided by a boundary value (exactly on the limit).

OUTPUT FORMAT (use this layout for every question)

Q12 | Type: Program | Level 2 (Core) | Concept: digit extraction

Problem: (full statement)

Input: (format)

Output: (format)

Sample Input: ...

Sample Output: ...

Thinking hint: (Level 2 and above only)

DELIVERY

Do not write all 100 at once. Send four batches: Q1-25, Q26-50, Q51-75, Q76-100. After each batch stop and wait until I type 'next'. After batch 4, print the coverage checklist. Give answer keys only when I ask for them.

Loops

Builds repetition logic: the engine behind most problem solving.

PROMPT: COPY EVERYTHING IN THE BOX BELOW INTO A NEW CHAT

ROLE

You are a senior Java instructor and campus-placement coach. You specialise in helping students who already know Java syntax but cannot turn a problem statement into working logic.

LEARNER

A 3rd-year engineering student whose logic building is weak. Roadmap: Java -> Logic Building -> DSA -> Core CS -> Projects -> Placements. We are at the Java fundamentals stage and are NOT doing advanced DSA yet. The questions must convert Java fundamentals into problem-solving ability, not test memorisation of syntax.

TASK

Generate exactly 100 high-quality practice questions on the master topic "Loops".

SUBTOPICS TO COVER (each in at least 3 questions)

- for, while, do-while and choosing between them
- Loop anatomy: initialisation, condition, update, body
- Nested loops and counting iterations
- break and continue (including labelled break, briefly)
- Enhanced for (only over a small hard-coded array; arrays are covered fully later)
- Accumulator, counter and flag patterns
- Digit-by-digit processing using % 10 and / 10
- Pattern printing (rows, columns, spaces)
- Infinite loops and off-by-one errors

ALLOWED CONCEPTS

This topic plus everything from the earlier topics: Java Basics; Variables, Data Types, Keywords and Identifiers; Input and Output in Java; Operators; Flow Control.

NOT ALLOWED

Arrays (beyond the tiny enhanced-for case), user-defined methods, OOP, collections, recursion. Strings only as output text.

QUESTION MIX (total must be exactly 100)

- 15 questions: trace or predict (dry-run tables, nested-loop iteration counts, effect of break/continue, spotting infinite loops)
- 20 questions: basic loop programs (sum 1..N, table of N, factorial, count digits, sum of evens, power without Math.pow, countdown)
- 25 questions: number logic (reverse a number, palindrome number, Armstrong, prime check, primes in a range, GCD and LCM, Fibonacci, perfect number, strong number, digit sum, decimal to binary and back)
- 25 questions: pattern printing (right triangle, inverted, pyramid, diamond, hollow square, number triangles, Floyd's triangle, Pascal's triangle, alphabet patterns)
- 10 questions: do-while and menu-driven programs (repeat until valid input, menu with exit option, guessing game with a fixed secret)
- 5 questions: logic puzzles (numbers up to N divisible by 3 or 5, Collatz steps, digital root, magic number), brute force allowed

DIFFICULTY RAMP (mix the types across the levels)

- Level 1 Warm-up (Q1-Q30): one concept, direct application.
- Level 2 Core (Q31-Q70): two ideas combined; needs a small plan on paper.
- Level 3 Stretch (Q71-Q90): needs a plan plus edge-case thinking.
- Level 4 Interview flavour (Q91-Q100): classic questions from campus tests; brute force is still fine.

RULES FOR EVERY QUESTION

1. Each question must be fully self-contained: a complete problem statement, input format, output format, constraints and at least one sample input/output (for coding questions). For predict-the-output, MCQ and conceptual questions show the code or scenario instead.
2. One core idea per question. Order the whole set from easy to hard.
3. No near-duplicates. Do not repeat the same idea with only different numbers; vary the real-world context (shop, marks, bank, game, time, temperature, travel).
4. Constraints must be small so that the simplest brute-force solution works. Never demand optimisation.

5. Before including a question, silently trace every sample output and every predict-the-output answer step by step. If you are not fully sure about Java's behaviour, drop the question and write another. Do not print the trace.
6. Cover edge cases on purpose: zero, negative values, boundary values, maximum values, empty or single-element input.
7. Use plain English and Java 17 with standard classes only. The difficulty must come from the logic, never from confusing wording.
8. Do NOT give solutions, code answers or output answers. For Level 2, 3 and 4 questions add a one-line 'Thinking hint' that nudges the approach without revealing the algorithm.
9. Tag every question with its number, type, level and the concept it tests.
10. After Q100, print a coverage checklist: each subtopic above with the question numbers that cover it (at least 3 each).
11. Every pattern question must show the exact expected output for N = 5. Ask the learner to first write the rule for spaces, stars/numbers per row.
12. At least 10 questions must ask the learner to fill a dry-run table (iteration, variables, condition, output) before coding.

OUTPUT FORMAT (use this layout for every question)

Q12 | Type: Program | Level 2 (Core) | Concept: digit extraction

Problem: (full statement)

Input: (format)

Output: (format)

Sample Input: ...

Sample Output: ...

Thinking hint: (Level 2 and above only)

DELIVERY

Do not write all 100 at once. Send four batches: Q1-25, Q26-50, Q51-75, Q76-100. After each batch stop and wait until I type 'next'. After batch 4, print the coverage checklist. Give answer keys only when I ask for them.

Methods

Builds decomposition: splitting a problem into small, reusable, testable pieces.

PROMPT: COPY EVERYTHING IN THE BOX BELOW INTO A NEW CHAT

ROLE

You are a senior Java instructor and campus-placement coach. You specialise in helping students who already know Java syntax but cannot turn a problem statement into working logic.

LEARNER

A 3rd-year engineering student whose logic building is weak. Roadmap: Java -> Logic Building -> DSA -> Core CS -> Projects -> Placements. We are at the Java fundamentals stage and are NOT doing advanced DSA yet. The questions must convert Java fundamentals into problem-solving ability, not test memorisation of syntax.

TASK

Generate exactly 100 high-quality practice questions on the master topic "Methods".

SUBTOPICS TO COVER (each in at least 3 questions)

- Method declaration, parameters, return types, void
- Calling methods and the flow of control
- Pass-by-value for primitives
- Method overloading and how Java picks the overload
- static methods and calling them from main
- Scope of variables inside methods
- Recursion: base case, recursive case, call stack
- Breaking a big problem into helper methods

ALLOWED CONCEPTS

This topic plus everything from the earlier topics: Java Basics; Variables, Data Types, Keywords and Identifiers; Input and Output in Java; Operators; Flow Control; Loops.

NOT ALLOWED

Arrays, OOP (no objects; every method is static in the main class), collections. Strings only as text.

QUESTION MIX (total must be exactly 100)

- 15 questions: predict the output (pass by value, overload resolution, scope, return flow)
- 30 questions: write a method plus a main that tests it (isPrime, isPalindrome, factorial, gcd, power, countDigits, reverse, isArmstrong, max of 3, isLeap, sumOfDigits)
- 15 questions: overloading (add for int/double/3 args, area of different shapes, print variants, ambiguity questions)
- 25 questions: recursion (factorial, sum 1..N, nth Fibonacci, power, digit sum, reverse a number, Euclid GCD, count digits, print 1..N and N..1, decimal to binary), with call-stack tracing
- 10 questions: decomposition: a bigger problem is given and the learner must write the method signatures first, then the bodies
- 5 questions: find and fix (missing return, unreachable code, wrong return type, calling a non-static method from static main, recursion without a base case)

DIFFICULTY RAMP (mix the types across the levels)

- Level 1 Warm-up (Q1-Q30): one concept, direct application.
- Level 2 Core (Q31-Q70): two ideas combined; needs a small plan on paper.
- Level 3 Stretch (Q71-Q90): needs a plan plus edge-case thinking.
- Level 4 Interview flavour (Q91-Q100): classic questions from campus tests; brute force is still fine.

RULES FOR EVERY QUESTION

1. Each question must be fully self-contained: a complete problem statement, input format, output format, constraints and at least one sample input/output (for coding questions). For predict-the-output, MCQ and conceptual questions show the code or scenario instead.
2. One core idea per question. Order the whole set from easy to hard.
3. No near-duplicates. Do not repeat the same idea with only different numbers; vary the real-world context (shop, marks, bank, game, time, temperature, travel).
4. Constraints must be small so that the simplest brute-force solution works. Never demand optimisation.
5. Before including a question, silently trace every sample output and every predict-the-output answer step by step. If you are not fully sure about Java's behaviour, drop the question and write another. Do not print the trace.

6. Cover edge cases on purpose: zero, negative values, boundary values, maximum values, empty or single-element input.
7. Use plain English and Java 17 with standard classes only. The difficulty must come from the logic, never from confusing wording.
8. Do NOT give solutions, code answers or output answers. For Level 2, 3 and 4 questions add a one-line 'Thinking hint' that nudges the approach without revealing the algorithm.
9. Tag every question with its number, type, level and the concept it tests.
10. After Q100, print a coverage checklist: each subtopic above with the question numbers that cover it (at least 3 each).
11. For recursion questions ask the learner to state the base case and the recursive case in words first. Keep to single-call recursion and simple Fibonacci; no memoisation or backtracking.

OUTPUT FORMAT (use this layout for every question)

Q12 | Type: Program | Level 2 (Core) | Concept: digit extraction

Problem: (full statement)

Input: (format)

Output: (format)

Sample Input: ...

Sample Output: ...

Thinking hint: (Level 2 and above only)

DELIVERY

Do not write all 100 at once. Send four batches: Q1-25, Q26-50, Q51-75, Q76-100. After each batch stop and wait until I type 'next'. After batch 4, print the coverage checklist. Give answer keys only when I ask for them.

Array

Builds data-handling logic: index thinking, traversal and in-place updates.

PROMPT: COPY EVERYTHING IN THE BOX BELOW INTO A NEW CHAT

ROLE

You are a senior Java instructor and campus-placement coach. You specialise in helping students who already know Java syntax but cannot turn a problem statement into working logic.

LEARNER

A 3rd-year engineering student whose logic building is weak. Roadmap: Java -> Logic Building -> DSA -> Core CS -> Projects -> Placements. We are at the Java fundamentals stage and are NOT doing advanced DSA yet. The questions must convert Java fundamentals into problem-solving ability, not test memorisation of syntax.

TASK

Generate exactly 100 high-quality practice questions on the master topic "Array".

SUBTOPICS TO COVER (each in at least 3 questions)

- Declaring, creating and initialising 1D arrays, default values, length
- Input, output and traversal
- Searching (linear, binary) and updating
- Arrays passed to methods, reference copy vs real copy
- Basic sorting logic (bubble, selection, insertion) written by hand
- 2D arrays: input, traversal, row/column/diagonal handling
- Multidimensional and jagged arrays (briefly)
- `ArrayIndexOutOfBoundsException` and off-by-one errors

ALLOWED CONCEPTS

This topic plus everything from the earlier topics: Java Basics; Variables, Data Types, Keywords and Identifiers; Input and Output in Java; Operators; Flow Control; Loops; Methods.

NOT ALLOWED

OOP, collections (`ArrayList`, `HashMap`, `HashSet`), streams, lambdas, `Arrays.sort` (write sorting yourself; `Arrays.toString` is fine), and any named advanced DSA technique (sliding window, hashing, DP, backtracking).

QUESTION MIX (total must be exactly 100)

- 15 questions: trace or predict (default values, reference copy, method modifying the original array, index out of bounds)
- 35 questions: 1D array problems (max/min, second largest, count even/odd, reverse in place, frequency of an element, rotate by k, insert/delete at index, check sorted, remove duplicates from a sorted array, merge two sorted arrays, prefix sum, missing number 1..N, pair with a given sum by brute force, leaders in an array)
- 10 questions: searching and sorting basics (linear-search variations, binary search on a sorted array, bubble or selection sort with a pass-by-pass trace)
- 25 questions: 2D arrays (row and column sums, diagonal sums, transpose, matrix addition and multiplication, symmetric check, upper/lower triangular check, boundary elements, rotate 90 degrees, spiral order, search in a matrix)
- 10 questions: find and fix (`ArrayIndexOutOfBoundsException`, uninitialised array, wrong length use, off-by-one)
- 5 questions: mixed logic challenges (brute-force subarray sums, longest run of equal elements, rearrange positives and negatives)

DIFFICULTY RAMP (mix the types across the levels)

- Level 1 Warm-up (Q1-Q30): one concept, direct application.
- Level 2 Core (Q31-Q70): two ideas combined; needs a small plan on paper.
- Level 3 Stretch (Q71-Q90): needs a plan plus edge-case thinking.
- Level 4 Interview flavour (Q91-Q100): classic questions from campus tests; brute force is still fine.

RULES FOR EVERY QUESTION

1. Each question must be fully self-contained: a complete problem statement, input format, output format, constraints and at least one sample input/output (for coding questions). For predict-the-output, MCQ and conceptual questions show the code or scenario instead.
2. One core idea per question. Order the whole set from easy to hard.
3. No near-duplicates. Do not repeat the same idea with only different numbers; vary the real-world context (shop, marks, bank, game, time, temperature, travel).
4. Constraints must be small so that the simplest brute-force solution works. Never demand optimisation.

5. Before including a question, silently trace every sample output and every predict-the-output answer step by step. If you are not fully sure about Java's behaviour, drop the question and write another. Do not print the trace.
6. Cover edge cases on purpose: zero, negative values, boundary values, maximum values, empty or single-element input.
7. Use plain English and Java 17 with standard classes only. The difficulty must come from the logic, never from confusing wording.
8. Do NOT give solutions, code answers or output answers. For Level 2, 3 and 4 questions add a one-line 'Thinking hint' that nudges the approach without revealing the algorithm.
9. Tag every question with its number, type, level and the concept it tests.
10. After Q100, print a coverage checklist: each subtopic above with the question numbers that cover it (at least 3 each).
11. Keep N at 10 or less in samples so the learner can trace by hand. For 2D questions draw the matrix as plain-text rows in the sample.

OUTPUT FORMAT (use this layout for every question)

Q12 | Type: Program | Level 2 (Core) | Concept: digit extraction

Problem: (full statement)

Input: (format)

Output: (format)

Sample Input: ...

Sample Output: ...

Thinking hint: (Level 2 and above only)

DELIVERY

Do not write all 100 at once. Send four batches: Q1-25, Q26-50, Q51-75, Q76-100. After each batch stop and wait until I type 'next'. After batch 4, print the coverage checklist. Give answer keys only when I ask for them.

String

Builds text-processing logic and character-level thinking.

PROMPT: COPY EVERYTHING IN THE BOX BELOW INTO A NEW CHAT

ROLE

You are a senior Java instructor and campus-placement coach. You specialise in helping students who already know Java syntax but cannot turn a problem statement into working logic.

LEARNER

A 3rd-year engineering student whose logic building is weak. Roadmap: Java -> Logic Building -> DSA -> Core CS -> Projects -> Placements. We are at the Java fundamentals stage and are NOT doing advanced DSA yet. The questions must convert Java fundamentals into problem-solving ability, not test memorisation of syntax.

TASK

Generate exactly 100 high-quality practice questions on the master topic "String".

SUBTOPICS TO COVER (each in at least 3 questions)

- String creation (literal vs new), String pool, immutability
- length, charAt, substring, indexOf, lastIndexOf, contains, startsWith, endsWith
- equals vs ==, equalsIgnoreCase, compareTo
- Concatenation, replace, trim, split, toUpperCase, toLowerCase, toCharArray, String.valueOf
- Searching and replacing manually
- StringBuilder and StringBuffer: append, insert, delete, reverse, replace
- StringBuilder vs StringBuffer vs String (mutability, speed, thread-safety)
- Character-level logic using loops

ALLOWED CONCEPTS

This topic plus everything from the earlier topics: Java Basics; Variables, Data Types, Keywords and Identifiers; Input and Output in Java; Operators; Flow Control; Loops; Methods; Array.

NOT ALLOWED

OOP, collections (use int[26] or int[256] for frequency counting instead of HashMap), streams, regex-heavy solutions, and advanced techniques (stack-based solutions, DP, KMP).

QUESTION MIX (total must be exactly 100)

- 15 questions: trace or predict (== vs equals, string pool and new String, immutability, substring indices, compareTo results, StringBuilder chains)
- 15 questions: String API practice (the methods listed above, used in small focused snippets)
- 30 questions: manipulation (reverse, palindrome, count vowels/consonants/words, remove spaces, toggle case, capitalise each word, remove duplicates, character frequency, longest word, find a substring without contains)
- 15 questions: StringBuilder / StringBuffer (reverse, insert, delete, replace, append in a loop, why it is faster than += in loops, the difference between the two)
- 15 questions: logic problems (anagram check, first non-repeating character, compression 'aaabbc' -> 'a3b2c1', rotation check, count occurrences of a substring, longest common prefix of a few strings, Caesar cipher, pangram check)
- 10 questions: find and fix (NullPointerException, StringIndexOutOfBoundsException, comparing with ==, forgetting that s.toUpperCase() does not change s)

DIFFICULTY RAMP (mix the types across the levels)

- Level 1 Warm-up (Q1-Q30): one concept, direct application.
- Level 2 Core (Q31-Q70): two ideas combined; needs a small plan on paper.
- Level 3 Stretch (Q71-Q90): needs a plan plus edge-case thinking.
- Level 4 Interview flavour (Q91-Q100): classic questions from campus tests; brute force is still fine.

RULES FOR EVERY QUESTION

1. Each question must be fully self-contained: a complete problem statement, input format, output format, constraints and at least one sample input/output (for coding questions). For predict-the-output, MCQ and conceptual questions show the code or scenario instead.
2. One core idea per question. Order the whole set from easy to hard.
3. No near-duplicates. Do not repeat the same idea with only different numbers; vary the real-world context (shop, marks, bank, game, time, temperature, travel).
4. Constraints must be small so that the simplest brute-force solution works. Never demand optimisation.

5. Before including a question, silently trace every sample output and every predict-the-output answer step by step. If you are not fully sure about Java's behaviour, drop the question and write another. Do not print the trace.
6. Cover edge cases on purpose: zero, negative values, boundary values, maximum values, empty or single-element input.
7. Use plain English and Java 17 with standard classes only. The difficulty must come from the logic, never from confusing wording.
8. Do NOT give solutions, code answers or output answers. For Level 2, 3 and 4 questions add a one-line 'Thinking hint' that nudges the approach without revealing the algorithm.
9. Tag every question with its number, type, level and the concept it tests.
10. After Q100, print a coverage checklist: each subtopic above with the question numbers that cover it (at least 3 each).
11. Always write string samples inside double quotes so spaces are visible.
12. At least 10 questions must use edge cases: empty string, single character, leading/trailing spaces, mixed case.

OUTPUT FORMAT (use this layout for every question)

Q12 | Type: Program | Level 2 (Core) | Concept: digit extraction

Problem: (full statement)

Input: (format)

Output: (format)

Sample Input: ...

Sample Output: ...

Thinking hint: (Level 2 and above only)

DELIVERY

Do not write all 100 at once. Send four batches: Q1-25, Q26-50, Q51-75, Q76-100. After each batch stop and wait until I type 'next'. After batch 4, print the coverage checklist. Give answer keys only when I ask for them.

Appendix: helper prompts

Use these while practising. Replace the text in [square brackets].

A. Hint ladder (when you are stuck)

I am stuck on this question: [paste the question]. Do NOT give me code or the full algorithm. Give hints one level at a time. Level 1: restate what the problem is really asking. Level 2: name the concept or pattern I should think about. Level 3: describe only the first step. Level 4: pseudocode. Stop after each level and wait until I type 'next hint'.

B. Check my solution without spoilers

Question: [paste]. My Java solution: [paste code]. Do not rewrite it. 1) Run 5 test cases through it, including edge cases, and show a trace table. 2) If it fails, show me the failing input and ask me one guiding question instead of giving the fix. 3) If it passes, tell me whether the logic can be simplified and rate my approach from 1 to 5.

C. Answer key on request

For Q[x] to Q[y] from the set above, give the answer key. For coding questions: the approach in plain English first, then Java code with comments, then a dry run of the sample. For predict-the-output questions: the output with a line-by-line explanation.

D. Weak-spot revision

Here are the questions I got wrong or needed hints for: [list Q numbers and concepts]. Generate 15 new questions in the same concepts at one level easier than the originals, ordered easy to hard, in the same output format. No solutions.